Impact Énergétique du Projet

Tout au long du projet, chaque nouvelle étape qui s'offrait à nous devait prendre en compte un facteur non négligeable en plus de la performance et de l'efficience : **l'impact énergétique**.

En tant que futurs ingénieurs, nous étions conscients de l'importance de nous habituer dès aujourd'hui à cette contrainte environnementale, bien qu'elle soit parfois peu abordée lors de l'apprentissage théorique des notions. Pour que notre compilateur soit moins "gourmand", on a cherché plusieurs astuces pour optimiser son fonctionnement. Dans ce bilan, on va vous expliquer ce qu'on a testé, les résultats qu'on a obtenus et comment on pourrait encore faire mieux.

1) Méthode d'Horner

Au niveau de l'extension, une attention particulière a été portée sur le nombre d'opérations nécessaires au calcul des fonctions trigonométriques. L'optimisation majeure repose sur l'utilisation de la **méthode de Horner**.

Grâce à la factorisation, cette méthode permet d'évaluer un polynôme de manière beaucoup plus efficace que la méthode naïve. Pour un polynôme de degré n :

$$P(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_n)\dots))$$

Listing 1 – Implémentation de la méthode de Horner en Deca

```
// Calcule cos(f) pour f dans [-pi/4, pi/4]
float _cos_horner(float f) {
    float f_2 = f * f;
    float a = -0.5;
    float b = 0.04166666;
    float c = -0.001388889;
    float d = 0.000024801;
    // Methode de Horner
    return 1.0 + f_2 * (a + f_2 * (b + f_2 * (c + f_2 * d)
       ));
}

// Calcule sin(f) pour f dans [-pi/4, pi/4]
float _sin_horner(float f) {
    float f_2 = f * f;
    float a = -0.1666667;
    float b = 0.008333333;
    float c = -0.000198413;
    float d = 0.000002756;
    // Methode de Horner
    return f * (1.0 + f_2 * (a + f_2 * (b + f_2 * (c + f_2
        * d))));
}
```

Comparée à une approche classique (calculant chaque puissance séparément), la méthode de Horner présente deux avantages majeurs : une réduction drastique des instructions de multiplication (`MUL`) et une précision accrue en limitant les erreurs d'arrondi machine.

2) Efficience de l'assemblage d'instructions

L'observation du fichier assembleur produit par notre compilateur révèle des pistes d'optimisation importantes concernant la communication avec la mémoire.

Listing 2 – Exemple de génération de code pour une assignation

```
; Declaration de la variable x
LOAD #0, R2
STORE R2, 2(GB)

; Assignation a x
LOAD 2(GB), R2
LOAD #4, R3

; Operation arithmetique binaire : +
ADD R3, R2
STORE R2, 2(GB)
```

On peut remarquer que le programme communique de manière intensive avec la RAM (via des instructions `LOAD` et `STORE` répétées). Bien que nous ayons pris en compte les contraintes de registres, un assemblage plus efficient aurait pu maintenir les variables fréquemment utilisées dans les registres `R2` à `R15` pour limiter les accès au bus mémoire, très énergivores.

3) Tests et procédé de fabrication

Pour l'exécution des tests, nous avons implémenté l'option d'exécution en parallèle du compilateur. Cette décision permet d'utiliser à leur plein potentiel les différents cœurs du CPU lors des phases intensives de validation.

En complément, l'option `-v` (vérification contextuelle seule) permet d'économiser les cycles CPU liés à la génération de code et à l'écriture de fichiers sur le disque lorsque seule la validité sémantique est testée. Enfin, la détection précoce des erreurs a permis de limiter les itérations de compilation inutiles.

4) Recherches extérieures et éthique

Conscients de l'impact environnemental massif des centres de données et de la consommation énergétique des modèles de langage (LLM), notre équipe a privilégié une lecture attentive de la documentation technique. Cette approche "low-tech" privilégie l'intelligence humaine plutôt que la sollicitation de sources extérieures.

5) Analyse des performances et bilan énergétique

Afin de mesurer l'impact réel de l'option de parallélisation (`-P`), nous avons soumis le compilateur à une charge de travail massive : la compilation simultanée de 11 fichiers comportant chacun 100 classes (soit 1100 définitions de classes et méthodes). Les mesures obtenues via `/usr/bin/time -v` mettent en évidence le compromis entre performance temporelle et efficacité énergétique.

Indicateur	**Séquentiel**	**Parallèle (`-P`)**	**Écart**
Temps réel (*Wall clock*)	0,87 s	0,59 s	**-32 %**
Travail CPU total (*User time*)	2,99 s	6,29 s	+110 %
Charge CPU moyenne	388 %	1200 %	+812 %
Consommation RAM (*Max RSS*)	282 MB	231 MB	**-18 %**
Changements de contexte involontaires	179	1087	+507 %

TABLE 1 – Impact de la parallélisation sur une charge de 1100 classes

Analyse du "Paradoxe de la Parallélisation" : On observe une accélération significative du temps de traitement réel (-32 %), permettant de libérer les ressources de la machine plus rapidement. Cependant, le travail processeur (*User time*) fait plus que doubler. Ce surplus d'énergie consommée s'explique par deux facteurs critiques :

— **L'Overscheduling :** La multiplication des *involuntary context switches* (+507 %) indique que le système d'exploitation doit ar-

bitrer une compétition intense entre les threads pour l'accès aux cœurs physiques. Chaque basculement consomme des cycles CPU pour sauvegarder et restaurer les registres, générant une dissipation thermique sans utilité directe pour la compilation.

— **La contention matérielle :** À 1200 % de charge CPU, les threads entrent en conflit pour l'accès au bus mémoire et aux caches partagés (L3), augmentant le temps de travail total nécessaire pour traiter la même quantité de données.

Efficience mémoire : Paradoxalement, le mode parallèle réduit l'empreinte RAM de 18 %. Cela suggère une gestion plus agressive du *Garbage Collector* de la JVM ou une durée de vie réduite des objets de l'arbre syntaxique (AST) en mémoire vive, ce qui limite la consommation électrique statique de la RAM.

Conclusion : L'option `-P` est un levier de productivité majeur mais coûteux en "Joules". Elle est optimale pour réduire le temps d'immobilisation d'une station de travail, mais le mode séquentiel reste préférable pour une compilation sobre sur batterie.

SaPour finir, n'oublions pas que 80 pourcents de notre groupe se déplaçait à l'école en vélo, tram, ou à pied ;).

Conclusion : Un équilibre entre performance et sobriété

Ce projet de compilation nous a permis de confronter les exigences de performance logicielle aux impératifs de sobriété énergétique. À travers nos expérimentations, nous avons identifié trois piliers fondamentaux pour une ingénierie logicielle durable :

1. **L'efficience du produit (Code IMA) :** L'optimisation de la gestion des registres et l'utilisation d'algorithmes mathématiques performants (comme la méthode de Horner pour l'extension Math) sont les leviers les plus directs. En minimisant les accès à la mémoire vive (RAM) au profit des registres processeur, nous réduisons le nombre de cycles d'horloge et la dissipation thermique par opération.
2. **L'efficience du procédé (Compilateur) :** Nos tests de parallélisation via l'option `-P` ont révélé un paradoxe instructif. Si le parallélisme augmente la productivité humaine en réduisant le temps d'attente, il engendre un surcoût énergétique significatif au niveau du processeur. La véritable sobriété réside donc dans le choix du mode de compilation adapté à la taille du projet traité.
3. **L'efficience méthodologique :** La réduction de notre dépendance aux ressources numériques externes (LLM, serveurs distants) au profit de la documentation technique locale et d'une validation sélective des tests constitue une forme de "sobriété numérique" essentielle à notre futur métier d'ingénieur.

En conclusion, bien que des optimisations supplémentaires soient envisageables (notamment via une analyse de l'espérance de vie des variables pour limiter encore davantage le *spilling*), notre compilateur actuel propose un compromis solide. Il ne se contente pas de transformer un code source en instructions machines ; il tente de le faire en respectant au mieux les ressources matérielles et énergétiques à sa disposition.